

The Best Programming Language for Tokenmaxxing

An Investigation of Coding Agent Behavior Across Programming Languages

Zixuan Wu
Northeastern University

Carolyn Jane Anderson
Wellesley College

Arjun Guha
Northeastern University

Abstract

Although coding agents are now very effective in a variety of programming languages, this paper first shows that the cost (in tokens) can vary significantly by programming language. We evaluate five recent models on programming problems in Python, Java, Rust, and OCaml. We carefully control for problem difficulty, and show that there can be stark variation in token consumption that is consistent across models. To understand why, we analyze both the structure and content of agent trajectories. First, we re-execute every intermediate solution and abstract each trajectory as a sequence of test-outcome vectors, then label the work between successive solutions. This reveals agents repeatedly producing noncompiling solutions in unfamiliar languages and revising solutions that already pass. Second, we analyze trajectory text, finding that agents plan solutions in code comments, distrust the provided tests in favor of inputs they invent, and sidestep unfamiliar target languages by prototyping in Python.

Our results show that by-language token efficiency is a metric that should be considered when benchmarking and developing multilingual agents, and, for the tokenmaxxer, a guide to the most expensive language to work in.

Let’s say you have a software engineer or AI researcher and you pay them \$500,000 a year. We do that all the time. That \$500,000 engineer, at the end of the year, I’m going to ask them, how much did you spend in tokens? If that person said, \$5,000, I will go ape... something else. If that \$500,000 engineer did not consume at least \$250,000 worth of tokens, I’m going to be deeply alarmed. — Jensen Huang (*All-In Podcast*)

1 Introduction

Over the past 1½ years, coding agents have gone from research idea (Yang et al., 2024) to tools

that are widely used by programmers. Although many programmers use agents by choice, others feel compelled to do so to meet corporate targets for AI adoption. There are even reports of organizations running internal leaderboards for token consumption. Following Goodhart’s Law (Strathern, 1997), some have resorted to *tokenmaxxing*, or giving agents tasks for the purpose of increasing LLM spending, without regard to whether it is worth the cost. We, the authors of this paper, think this is definitely pointless and possibly unethical. But, for those who must tokenmax, we present a principled way to do so: work in the programming language that makes the agent consume the most tokens. Naively, one might think that this is the language in which programs are the longest, but this isn’t the case with agents that reason and self-correct. What matters is not the number of tokens in the final solution, but the amount of reasoning and revision needed to get to the solution.

More seriously, the goal of this paper is to study how the behavior of software-engineering agents varies by programming language. We evaluate four programming languages (Java, OCaml, Python, and Rust) and five models on a 100-problem subset of LiveCodeBench, gathering 2,000 agent trajectories for analysis.

Our basic finding is that the choice of programming language has a significant effect on output tokens produced by the agent, even after carefully controlling for problem difficulty. The effect is most pronounced for OCaml across all models, but is significant for Rust and Java too. Models are most efficient when they work in Python.

However, we go beyond counting tokens and analyze agent trajectories in several ways. First, we evaluate every intermediate solution that the agents produce, which allows us to study an agent’s progression on the unit tests for a problem. This allows us to observe and quantify several behaviors:

agents repeatedly struggling with the same compile error; agents that get stuck revising a solution even when all tests pass; and so forth. Second, we label the actions in every trajectory with an LLM using a 12-category taxonomy that we develop. This allows us to ask what an agent is doing as it works through a problem. These labels allow us to determine the turns spent on bug fixes, optimizations, exploration, refactors, and more. Finally, we analyze the text the agents produce as they work, which reveals specific behaviors: agents sometimes plan in comments before writing code; they sometimes second-guess and re-derive already-passing solutions; they can distrust the provided tests in favor of inputs they invent; and they can sidestep an unfamiliar language by prototyping in Python first.

Contributions We make the following contributions. (1) We study how token cost in agentic coding varies across programming languages, controlling for problem difficulty. (2) We find that, at comparable success rates, lower-resource languages cost significantly more than Python. (3) We introduce a novel analysis approach that involves testing every intermediate solution that an agent writes in an external test harness, which allows us to analyze agents’ test progression. (4) We classify the steps between intermediate solutions as semantic actions, revealing specific agent behaviors and where they fall on the path to a solution.

These findings show how the choice of programming language shapes an agent’s token cost and which behaviors drive the difference between languages, offering insight toward building more efficient agents across programming languages. We plan to release all code and data publicly upon publication.

2 Background and Related Work

2.1 Multilingual Code Evaluation

Many benchmarks evaluate how the choice of programming language affects models’ code generation performance. The earliest benchmarks translated single-function code completion tasks into other languages (Cassano et al., 2023; Athiwaratkun et al., 2022). Multi-SWE-bench and SWE-bench Multilingual (Zan et al., 2026; Yang et al., 2025b) evaluate agents on their ability to solve real-world issues in large code repositories. The tasks that we study lie between code completion and repository-level tasks. We do not introduce

a new benchmark, but use LiveCodeBench in a multi-language context to study agent trajectories.

2.2 Agentic Coding and Process Cost

Beyond correctness, prior work measures several notions of code-generation cost. One line studies the efficiency of the generated program itself—its runtime and memory across languages (Fan et al., 2025; Qing et al., 2025). Another studies the token cost of the program’s surface representation, where formatting and verbose recurring patterns add tokens without functional benefit (Pan et al., 2026; Sun et al., 2025). In an agentic setting, however, multi-turn reasoning and tool use can cost far more than the final program. Existing work on this process cost focuses almost entirely on Python (Bai et al., 2026). We instead make the target language the independent variable: process cost is not surface cost, and the language in which an agent spends the most tokens need not be the one whose programs are longest.

Prior work documents particular sources of agent inefficiency, including continuing to generate after finding a passing solution (Solovyeva and Castor, 2026) and looping when stuck (Fan et al., 2025). Related methods stop trajectories early or prune redundant work (Guo et al., 2026; Gao and Peng, 2026; Xiao et al., 2026; Yang et al., 2025a). On less familiar languages, agents may instead prototype and debug in Python before translating to the target language (Sharma et al., 2026). Recent analyses open up the trajectory by decomposing token cost by development phase (Salim et al., 2026) or agent action (Bouzenia and Pradel, 2025; Liu et al., 2026). Our analysis likewise resolves a trajectory into phases, but grounds those phases in changes in externally verified test outcomes between successive program snapshots. This lets us localize behaviors—including stalling, reworking a passing solution, and sidestepping through Python—to particular stages of solution progress.

3 Methods

We first present how we setup the multi-language environment, the models we use, how we prompt agents, and how we run our experiments.

3.1 Choice of Agents, Models, and Languages

We use mini-swe-agent (Yang et al., 2024), a customizable agent that supports any model and has a stable interface. It is also an established baseline

Table 1: We run agents in a container based on Ubuntu 24.04. The container has the language versions shown below, along with optional packages that may help agents solve programming tasks. We build and run agents’ solutions with the compile and run commands shown below, and we provide these commands to the agent in the prompt as part of the task.

Language	Version	Extra Packages	Compile command	Run command
Python	3.12.3	numpy, pandas, scipy, sympy	—	python3 solution.py
OCaml	5.4.1	core, base, zarith	dune build ./solution.exe	dune exec ./solution.exe
Java (OpenJDK)	21.0.11	—	javac Solution.java	java Solution
Rust	1.96	—	cargo build --release	./target/release/solution

agent that is widely used (Merrill et al., 2026; Zhou et al., 2026; Li et al., 2026; Zhang et al., 2026).

We work with five models: QWEN3.6-35B-A3B, GEMMA-4-26B-A4B, GLM-5.2, GPT-5.5, and CLAUDE-SONNET-4.6 (Qwen Team, 2026; Google, 2026; GLM-5-Team et al., 2026; Anthropic, 2026; OpenAI, 2026). We run the open-weight Qwen and Gemma models locally and access the open-weight GLM model via the Hugging Face Inference API; all three are amenable to reproducibility. We also evaluate the proprietary GPT-5.5 and Sonnet models that power Codex and Claude Code, respectively. We enable reasoning for all models. Additional implementation details and hyperparameters are provided in §A.

We test agents in Python, Java, Rust, and OCaml. Python is the language used most widely to benchmark models’ programming abilities. Although Java is not popular in ML research, it remains widely used. Rust and OCaml help illuminate agent behavior on languages that are not as well-represented in pretraining data.¹

3.2 Environment Configuration

We run agents in a containerized environment that includes the standard toolchain for each programming language, and common packages (Table 1). Table 1 also shows how we compile and run solutions to verify agents’ results. For OCaml and Rust, the environment includes a working configuration for Dune and Cargo, so that the agent does not need to spend turns configuring the compiler. The agent is thus free to edit the Dune/Cargo configuration, or write a solution that spans several files.

At runtime, we limit the agent to 40 turns and 10 minutes of execution for each problem. We also

impose a 256MB memory limit on each container, and as described below, take steps to prevent code execution from causing timeouts. §A describes the host machine configuration.

3.3 The MiniLCB Benchmark

LiveCodeBench (Jain et al., 2025) consists of problems from programming contest websites and is typically used to evaluate models in a single attempt. Version 6 contains 880 problems, of which 381 have Python starter code; the remaining 499 are effectively language-neutral and test solutions over standard I/O. Agnostics (Boruch-Gruszecki et al., 2026) and MultiLCB (Ivanova et al., 2026) use this observation to evaluate models across languages with per-language compile-and-run configurations. We rely on the same observation to construct an agent benchmark.

With five models, four programming languages, and these 499 language-agnostic tasks, we have the opportunity to run 9,980 agent sessions. To conserve tokens and budget, we instead randomly sample 100 tasks (yielding 2,000 sessions), and we refer to this subset as **MiniLCB**. Test cases per problem range from 2 to 45, with a median of 22.

In standard LiveCodeBench evaluation, test cases run in an external harness to validate a model’s single solution offline. In MiniLCB, *we make all tests and the test harness visible to the agent*. Specifically, we present each problem as a small repository: a `tests/` directory holding all input/output pairs and a `test.sh` script that runs the agent’s solution and reports successes and failures on individual tests.² The prompt explicitly states³:

²We put some effort into building a robust test script that works even for a weak model/agent. We set timeouts on each test, to help agents recover when they write non-terminating programs. The test script takes care of recompilation when needed, in case the agent forgets to compile. Our first version only printed failure and used exit code 0 to indicate success. We found that Qwen and Gemma had trouble with this format, so we switched to printing PASS and FAIL.

³See §F for the complete prompt template.

¹We are assuming that the pretraining corpora of proprietary models has a distribution similar to the few public pretraining code corpora available for inspection, such as The Stack v2 (Lozhkov et al., 2024). After deduplication, The Stack v2 has around 200GB of Java and of Python, but only 12GB of Rust and even less OCaml.

To test your program, run exactly `./test.sh`
 [...] These are the only tests I care about.

We believe that this setup is a realistic way to study agent behavior: in everyday use, programmers don’t hide their tests from agents. Instead, programmers direct their agents to keep working until all tests pass. It is this type of task that MiniLCB simulates, and we get to observe how agents iterate on their solutions when they have test feedback available. With this configuration, there is a risk that the agent will cheat, but we only find cheating in 2 out of all traces (§B).

There is also a risk that an agent will timeout while running test cases, if it writes an inefficient or non-terminating solution. To mitigate this, the `test.sh` script imposes a 10-second timeout on each test case. An agent run terminates when the agent submits its solution, exhausts its 40-turn budget, or exceeds the 600 s wall-clock limit.

4 Results

For a complex task, agents may engage in multiple turns of reasoning, interleaving different kinds of behavior. Thus, when looking at agent behavior across languages, we can look both at differences in the turns they take, and in the content of those turns. We explore both aspects below.

4.1 Token Cost and Accuracy by Language

When comparing agent behavior across programming languages, the most basic question is how many tokens are consumed to solve a problem, which we measure as the total output tokens generated over the trajectory (§E).

Median token cost and accuracy In Figure 1 we report the median output token usage and accuracy by language and model. For the models we explore, accuracy on MiniLCB is not affected very much by language, with the notable exception that Gemma performs much worse on OCaml. However, output tokens vary substantially by language. We report medians because token costs are heavy-tailed: a few hard problems inflate the mean. For all models, agents produce significantly more tokens when writing OCaml, and the fewest tokens when writing Python. We consider token usage across all traces, and not just the succeeding traces. We believe this is realistic: if an agent runs for hours and fails, you don’t get a refund.

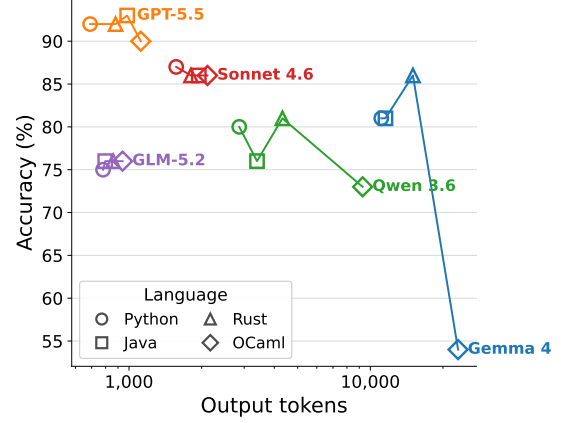


Figure 1: Accuracy versus median output token usage for each model and programming language. The output token axis is shown on a logarithmic scale.

Controlling for problem difficulty A limitation of the medians above is that they conflate programming-language difficulty with problem difficulty, both of which affect output tokens. To isolate the effect of language, we fit *mixed-effects models* for each LLM: a linear model that assigns a baseline token cost to each problem, so the language coefficients reflect cost differences *on the same problem*. We treat language as a fixed effect with Python as the reference level, and the choice of problem as a random effect. The intraclass correlation (ICC) measures what fraction of token-cost variance is explained by problem identity; the high values (0.80–0.97) confirm that problem difficulty dominates, justifying why we control for it.

The mixed-effect model allows us to quantify the token penalty (or advantage) for choosing a language other than Python, after controlling for problem difficulty (Tables 2 and 7). Across all models, OCaml traces consume significantly more tokens than Python traces ($1.28\text{--}1.69\times$ vs. Python, $p < 0.001$). The penalty for using Rust or Java is model-dependent, but not significant for Gemma. Overall, these results strongly suggest that token-maxxers should not use Python, and of these languages, OCaml is best for tokenmaxxing.

4.2 Where Do All Those Tokens Go?

In the rest of this paper, we go beyond accuracy and try to shed light on what these agents are doing with all of these tokens. We present the results for open-weight models below; the corresponding results for proprietary models are provided in §D.

Table 2: Mixed-effects model results: multiplicative output token cost relative to Python, after controlling for problem difficulty. OCaml has a significant penalty with all models; Java and Rust effects are model-dependent.

Model	Contrast	Extra toks	Significance
Gemma	Java vs. Python	$0.85\times$	$p = 0.114$
	Rust vs. Python	$1.07\times$	$p = 0.480$
	OCaml vs. Python	$1.44\times$	$p < 0.001$
	ICC	0.80	—
Qwen	Java vs. Python	$1.21\times$	$p < 0.05$
	Rust vs. Python	$1.20\times$	$p < 0.05$
	OCaml vs. Python	$1.69\times$	$p < 0.001$
	ICC	0.89	—
GLM	Java vs. Python	$1.34\times$	$p < 0.01$
	Rust vs. Python	$1.57\times$	$p < 0.001$
	OCaml vs. Python	$1.54\times$	$p < 0.001$
	ICC	0.73	—

Table 3: We run all intermediate solutions that the agent generates and report the number of tokens (turns) to the first solution that passes all tests, and the tokens (turns) spent after that solution.

Model	Language	Tokens (Turns) to Solution	Tokens (Turns) after Solution
Gemma	Python	1743 (1)	3554 (3)
	Java	2454 (1)	4546 (2)
	Rust	2026 (1)	7311 (3)
	OCaml	1760 (2)	1173 (2)
Qwen	Python	1755 (1)	283 (3)
	Java	2138 (1)	272 (2)
	Rust	2409 (3)	370 (4)
	OCaml	3177 (5)	405 (4)
GLM	Python	758 (1)	117 (2)
	Java	729 (1)	97 (1)
	Rust	1574 (1)	122 (2)
	OCaml	1374 (3)	109 (2)

4.2.1 Tokens after the first correct solution

One surprisingly common behavior that we observe is that agents continue to refine their solutions after they get it right, sometimes even after all tests pass. We study this behavior by splitting successful trajectories into two pieces: the turns before and after the agent produces the first correct solution.⁴

Table 3 shows the median number of tokens that each model needed to produce a working solution for each language and the median number of additional tokens produced afterward. Our prompt directs the model to print a sentinel message when it is done, which requires a few tokens. But, the smaller models (Gemma and Qwen) produce hun-

⁴We test intermediate solutions after the agent concludes execution. Thus we identify all correct solutions. The agent may not test every solution.

dreds or thousands more tokens after they find their first working solution.

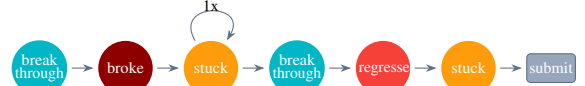
4.2.2 Progression on provided unit tests

We now analyze trajectories by running the test suite on every intermediate solution that the agent writes to disk and record the list of test outcomes. This allows us to abstract each trajectory to a list of *solution snapshots*, $s_0, s_1, \dots, s_n$, where each s_i is a list of test outcomes. We then classify each consecutive pair (s_{i-1}, s_i) based on how the test outcomes change from s_{i-1} to s_i :

- *breakthrough*: some tests were failing and all now pass;
- *stay*: all tests passed before and after, i.e., model rewrote a correct solution);
- *improved*: some failing tests now pass (but not all);
- *regressed*: some passing tests now fail (but not all fail);
- *churn*: some passing tests now fail, and failing tests now pass (improvement and regression);
- *broke*: some tests were passing, but now all fail, e.g., a compile error
- *stuck*: some tests fail, and no change in test outcomes

We build a graph where nodes represent these outcome changes, as well as special nodes for the start state, a submission (which may be wrong!), or reaching a resource limit.

In Figure 2, we show the graphs for Gemma and GLM on Python and OCaml. (The corresponding GPT-5.5 graphs appear in §D.) These graphs allow us to quickly discover interesting behavior in the models. For example, Gemma has exactly one *breakthrough*→*broke* edge across all Python problems in a trajectory that has this shape:



Examining it reveals performance anxiety: despite the breakthrough, the model’s reasoning text states:

seems to be correct. However, it might be too slow for the given constraints—let’s optimize

Unfortunately, the optimization fails all tests. It corrects it (the second breakthrough), then regresses, and finally submits broken code.

4.2.3 Text analysis of snapshot spans

The test outcomes of the solution snapshots show stages in the agent’s progress, but they do not reveal

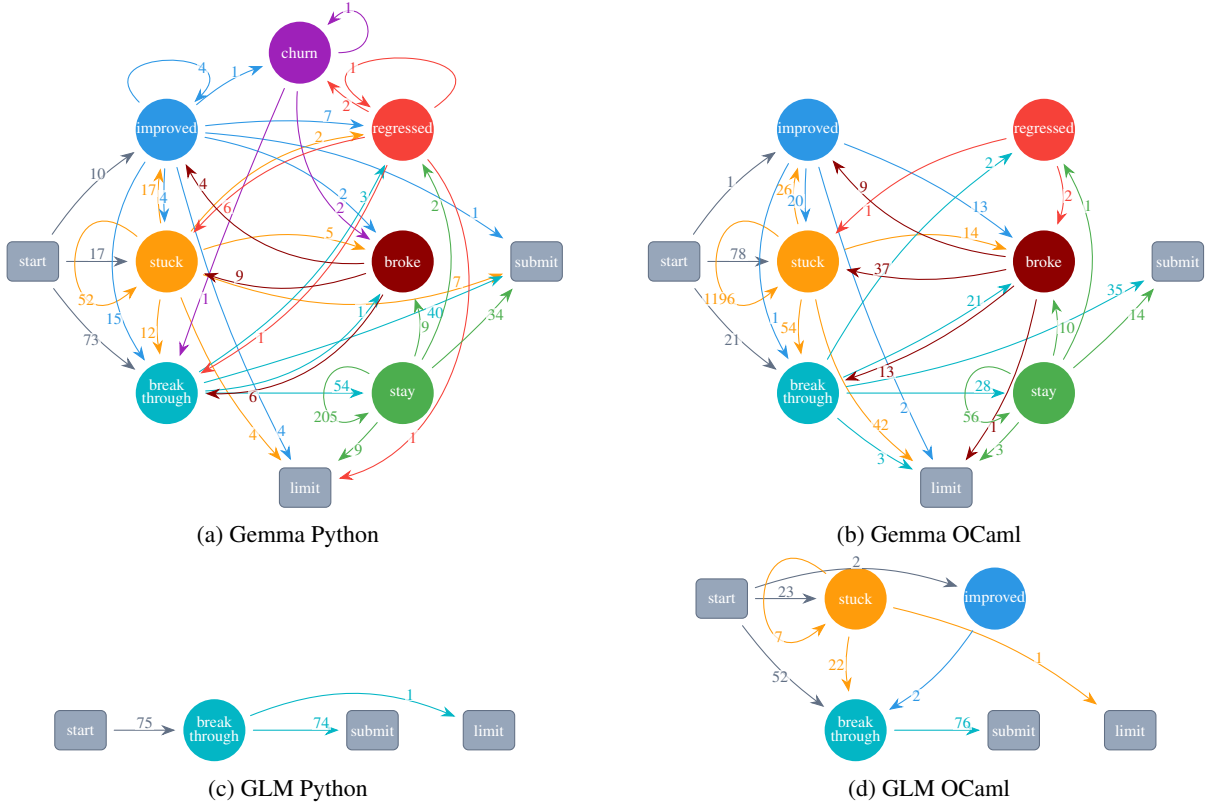


Figure 2: We test every solution that the agent writes to the solution file, which allows us to abstract each trajectory into a list of *solution snapshots* with test outcomes. We then classify each consecutive pair of test outcomes based on how outcomes change from the previous snapshot to the current snapshot. We describe the outcomes labels in detail in §4.2.2. The *breakthrough* and *stay* nodes are those where all tests pass in the snapshot, and all other colored nodes are cases where some tests fail. We consolidate trajectories for model-language pairs into individual graphs to see how models behave on each language.

what kinds of actions the agent takes during these stages. A natural unit for analysis is the turn, but turns are not semantically coherent: an agent often needs several turns to accomplish a one action: running a test, reading the output, are three turns that make one logical bugfix action. We therefore analyze at the level of *snapshot spans*. A span consists of everything the agent does as it moves between two consecutive solution snapshots: the resulting solution snapshot, the unified diff to the previous snapshot, the change in test outcomes, and the agent’s reasoning, bash command tool calls, and outputs up to and including the ending write.

We develop a taxonomy of twelve span labels by hand-examining a sample of trajectories and iterating until the labels were mutually exclusive and covered the full range of behaviors. We then labeled all spans by prompting an LLM judge (GPT-5-Mini) with the taxonomy and the span content:

- *explore*: working out a tentative approach — stubs, comments, or exploratory scripts;

- *implement*: commits real algorithmic logic, building on the current approach; prior code is kept and extended;
- *rewrite*: abandons the current approach and rewrite with a new algorithm;
- *execute plan*: fills in stubs or #TODO placeholders from the prior snapshot with real logic;
- *fix bug*: named cause, focused patch — the agent identified the bug and targets it directly;
- *debug failure*: cause unknown — speculative tweaks, debug prints, or repeated attempts without a named cause;
- *infra confusion*: struggling with scaffolding rather than the algorithm: wrong file conventions, misread test output, or build plumbing;
- *revert*: returns to a previously seen solution, discarding recent changes;
- *verify*: confirms correctness with no code change, before final submission;
- *edge case*: proactively hardens a specific input class with no failing test pointing at it;

Model	Lang	Implement	Explore	Rewrite	Execute plan	Fix bug	Debug failure	Infra confusion	Revert	Verify	Edge case	Optimize perf.	Cosmetic refactor
Gemma	java	31%	8%	7%	—	8%	4%	1%	—	26%	2%	4%	7%
	ocaml	17%	6%	2%	1%	16%	15%	5%	23%	7%	1%	—	5%
	python	25%	8%	4%	2%	8%	3%	1%	4%	19%	1%	10%	16%
	rust	27%	10%	5%	1%	8%	2%	1%	3%	23%	3%	4%	14%
Qwen	java	36%	5%	8%	—	16%	7%	1%	—	27%	—	—	—
	ocaml	19%	6%	5%	1%	37%	8%	12%	—	11%	—	—	2%
	python	35%	4%	8%	—	14%	4%	—	—	29%	—	4%	1%
	rust	36%	4%	8%	—	15%	3%	1%	—	29%	1%	—	2%
GLM	java	50%	1%	—	—	1%	—	—	—	49%	—	—	—
	ocaml	41%	—	—	—	13%	1%	4%	—	41%	—	—	1%
	python	49%	—	—	—	1%	—	1%	—	49%	—	—	—
	rust	48%	1%	—	—	4%	—	—	—	47%	—	—	—

Table 4: Percentage of spans assigned each label, for every (model,language) pair. GLM is mostly *implement* + *verify*; debugging labels concentrate in OCaml; post-success labels are Gemma-dominated.

- *optimize perf*: improves time/memory;
- *cosmetic refactor*: renaming, reformatting, or commenting with no behavioral change.

For example, the following three-turn span is labeled *fix bug*:

Assistant: Let me test this optimized solution with the sample cases first, then run the full test suite.

Assistant: The output is wrong. Let me debug...

Assistant: I see the bug ... Let me fix:

The next two-turn span is an *implement*, where Qwen first solves the problem in Python and then translates to OCaml:

Assistant: The v3 formula passes all 31 tests. Let me verify it more carefully and then implement it in OCaml:
cat << 'PYEOF' > analyze7.py
...

Assistant: Now let me implement the solution in OCaml:

For every model–language pair, we report the breakdown of span labels in Table 4. Across models, error-recovery labels—*fix bug*, *debug failure*, *infra confusion*, and *revert*—are disproportionately associated with OCaml, suggesting that solving OCaml problems requires more iterative correction. Post-success labels (*optimize perf*, *cosmetic refactor*) are most prevalent for Gemma, consistent with its large post-solution token costs in Table 3.

4.3 How Do Agents Get (Un)Stuck?

We can use the span labels (§4.2.3) to ask how agents get unstuck on problems, using Gemma on OCaml as an example. This model-language pair has 1,196 self-loops on the *stuck* state, dominated

by *revert* (31%): the agent rewrites a byte-identical solution that it has already tried, oscillated between 2-3 broken attempts. The agent is fighting OCaml syntax, not the algorithm: 92% of failed snapshots are compile-time errors and 8% are wrong-answer failures. The model rarely gets far enough to learn whether its algorithm is correct. Escape, when it comes, is a *language learning event*: *fix bug* dominates breakthrough at 60% (Table 5), the moment the agent finally names a concrete OCaml-specific error and the unchanged algorithm immediately passes. Python breakthroughs, by contrast, are spread across *implement*, *execute plan*, and *optimize perf*—there is no single bottleneck to escape.

On task abc311_d, for instance, Gemma loops on stuck 14 times before escape:

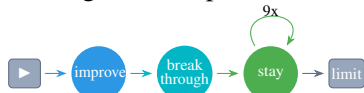


Gemma keeps the same algorithm throughout but cannot get it to compile, cycling on the same OCaml type errors. Turns 8–14 oscillate between two earlier attempts, re-emitting byte-for-byte identical code, until it finally names the error and the unchanged algorithm compiles and passes all tests.

4.4 Cosmetic Refactoring

As a second case study, we examine how Gemma revises perfect solutions (*stay* loops). Figure 2 shows 205 stay self-loops in Python, the most of any language. These loops are unnecessary polishing: *cosmetic refactor* and *optimize perf* account for half the edges into STAY (Table 6). For example, Gemma solves arc194_e in Python on Turn

2, but then second-guesses its solution and spends remaining turns on polish before a timeout:



Assistant: Turn 3/12. The logic was not quite right. I need to rewrite. ...

Assistant: Turn 9/12. My current code seems to be working on these samples. Is there any other condition? ...

Assistant: Turn 12/12. Okay, the current solution seems to be working for the sample cases. “Wait, the input could be quite large” ...

Turn 3 reformulates the correct solution, Turn 9 adds only inline comments, and Turn 12 changes the recursion depth, even though the solution is not recursion. It hits the timeout and never submits.

4.5 Text-Level Analysis

The case studies above analyzed the *structure* of trajectories—which nodes agents visit and how they move between them. We now move beyond the graph to analyze the *content*: the text agents produce as they work through a problem.

Planning in code comments Models sometimes solve problems in comments before writing code. E.g., on abc359_d, Gemma spends two turns deriving a DP solution in comments, before translating it into working code. On arc194_a, Qwen writes a 35-line Java solution with 3,439 lines of comments.

Solve in Python, then translate. Models sometimes solve problems in Python before translating the solution to an unfamiliar target language. E.g., to solve abc359_c in OCaml, Qwen first takes 12 turns to write a Python solution, including a translated test harness. Similarly, to solve arc194_e in Rust, GLM spends 11 turns in Python testing mathematical invariants against a brute-force checker before writing a Rust solution that works immediately. It then returns to Python to stress-test the solution on a large inputs before submitting.

Second-guessing in reasoning. Reasoning traces show that models frequently reconsider their approach. Gemma and Qwen reason very reflectively, frequently using words such as *wait* and *actually* (7.70 and 7.72 occurrences per turn, respectively), alongside frequent mentions of correctness (3.89 and 1.70) and efficiency (2.02 and 1.65). GLM shows the same tendency more moderately (4.27 second-guessing, 1.65 correctness, 0.87 efficiency

mentions per turn). These keywords appear in both affirmative and negative contexts (e.g., “*correct*” and “*incorrect*”), suggesting that the models repeatedly reevaluate whether their current solution is correct. The trajectory example in §4.4 illustrates this behavior: after producing a solution that retrospectively passes all tests, Gemma remarks, “Wait, the input could be quite large,” and repeatedly rewrites the solution.

Testing and self-examination. Gemma and Qwen often rewrite solutions without running provided tests: over 70% of their snapshot spans into STAY are bare solution writes with no preceding test execution. Instead, they frequently challenge their solutions by replaying the sample inputs or constructing their own test cases.

For example, on arc194_e Gemma repeatedly revises the solution using tests that it writes itself. However, it continues to do so even after it produces a solution that passes all provided tests—unfortunately, it never runs the provided tests on this problem despite directions to do so. This is in contrast to GLM, uses the provided tests to stop much earlier.

5 Conclusion

We examine how the choice of programming language affects the behavior of coding agents. Our most basic finding is that token consumption can vary significantly by programming language, and agents can spend far more tokens on a lower-resource language than higher-resource languages.

What is more revealing is what the agent is doing with its tokens on each language. We investigate this by abstracting agent trajectories into trajectories of intermediate solutions and their test results. This analysis exposes a range of token waste patterns—agents polishing solutions after all tests pass, and fighting a low-resource language’s syntax rather than improving the solution’s algorithm—and also more deliberate strategies, such as planning in code comments and prototyping in Python before translating to an unfamiliar target. We hope these findings point the way toward designing agents that work more efficiently across all programming languages.

Limitations

Our study covers four programming languages and five models; the behaviors we identify may not generalize to other models, and the language-specific

effects we observe may change as future models improve pretraining coverage of lower-resource languages. MiniLCB consists of self-contained competitive-programming problems with visible test cases, solved using a single agent scaffold (mini-swe-agent) with a fixed prompt, turn limit, and timeout. Our findings may therefore not generalize to repository-scale software engineering tasks, or to other agent harnesses.

Ethical Considerations

Our paper title is a joke. Our intent is to understand why agents tokens in low-resource programming languages so that more efficient multilingual agents can be built. We discourage using these findings to deliberately increase token consumption.

References

- All-In Podcast. Jensen Huang: Nvidia’s future, physical ai, rise of the agent, inference explosion, ai pr crisis.
- Anthropic. 2026. Claude Sonnet 4.6 System Card. <https://www-cdn.anthropic.com/78073f739564e986ff3e28522761a7a0b4484f84.pdf>. Accessed: 2026-06-17.
- Ben Athiwaratkun, Sanjay Krishna Gouda, Zijian Wang, Xiaopeng Li, Yuchen Tian, Ming Tan, Wasi Uddin Ahmad, Shiqi Wang, Qing Sun, Mingyue Shang, and 1 others. 2022. Multi-lingual evaluation of code generation models. *arXiv preprint arXiv:2210.14868*.
- Longju Bai, Zheming Huang, Xingyao Wang, Jiao Sun, Rada Mihalcea, Erik Brynjolfsson, Alex Pentland, and Jiaxin Pei. 2026. How do ai agents spend your money? analyzing and predicting token consumption in agentic coding tasks. *arXiv preprint arXiv:2604.22750*.
- Aleksander Boruch-Gruszecki, Yangtian Zi, Zixuan Wu, Tejas Oberoi, Carolyn Jane Anderson, Joydeep Biswas, and Arjun Guha. 2026. Agnostics: Learning to code in any programming language via reinforcement with a universal learning environment. In *International Conference on Learning Representations (ICLR)*.
- Islem Bouzenia and Michael Pradel. 2025. *Understanding software engineering agents: A study of thought-action-result trajectories*. In *2025 IEEE/ACM 40th International Conference on Automated Software Engineering, ASE ’25*.
- Federico Cassano, John Gouwar, Daniel Nguyen, Sydney Nguyen, Luna Phipps-Costin, Donald Pinckney, Ming-Ho Yee, Yangtian Zi, Carolyn Jane Anderson, Molly Q Feldman, Arjun Guha, Michael Greenberg, and Abhinav Jangda. 2023. MultiPL-E: A scalable and extensible approach to benchmarking neural code generation. *IEEE Transactions on Software Engineering*. ArXiv:2208.08227.
- Zhiyu Fan, Kirill Vasilevski, Dayi Lin, Boyuan Chen, Yihao Chen, Zhiqing Zhong, Jie M Zhang, Pinjia He, and Ahmed E Hassan. 2025. Swe-effi: Re-evaluating software ai agent system effectiveness under resource constraints. *arXiv preprint arXiv:2509.09853*.
- Pengfei Gao and Chao Peng. 2026. *More with less: An empirical study of turn-control strategies for efficient coding agents*. In *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering, ICSE ’26*.
- GLM-5-Team, :, Aohan Zeng, Xin Lv, Zhenyu Hou, Zhengxiao Du, Qinkai Zheng, Bin Chen, Da Yin, Chendi Ge, Chenghua Huang, Chengxing Xie, Chenzheng Zhu, Congfeng Yin, Cunxiang Wang, Gengzheng Pan, Hao Zeng, Haoke Zhang, Hao-ran Wang, and 168 others. 2026. *GLM-5: from vibe coding to agentic engineering*. *Preprint*, arXiv:2602.15763.
- Google. 2026. Gemma 4: Byte for byte, the most capable open models. <https://blog.google/innovation-and-ai/technology/developers-tools/gemma-4/>. Accessed: 2026-06-17.
- Yaoqi Guo, Ying Xiao, Jie M. Zhang, Mark Harman, Yiling Lou, Yang Liu, and Zhenpeng Chen. 2026. EET: Experience-driven early termination for cost-efficient software engineering agents. In *Findings of the Association for Computational Linguistics: ACL 2026*.
- Maria Ivanova, Pavel Zadorozhny, Rodion Levichev, Ivan Petrov, Adamenko Pavel, Ivan Lopatin, Alexey Kutalev, and Dmitrii Babaev. 2026. *Multi-LCB: Extending livecodebench to multiple programming languages*. In *The Fourteenth International Conference on Learning Representations*.
- Naman Jain, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. 2025. Livecodebench: Holistic and contamination free evaluation of large language models for code. In *International Conference on Learning Representations*, volume 2025, pages 58791–58831.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. *Efficient memory management for large language model serving with pagedattention*. In *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP ’23*, page 611–626, New York, NY, USA. Association for Computing Machinery.
- Han Li, Letian Zhu, Bohan Zhang, Rili Feng, Jiaming Wang, Yue Pan, Earl T Barr, Federica Sarro, Zhaoyang Chu, and He Ye. 2026. Contextbench: A benchmark for context retrieval in coding agents. *arXiv preprint arXiv:2602.05892*.

- Shuyang Liu, Yang Chen, Rahul Krishna, Saurabh Sinha, Jatin Ganhotra, and Reyhaneh Jabbarvand. 2026. [Process-centric analysis of agentic software systems](#). *Proceedings of the ACM on Programming Languages*, 10(OOPSLA1):1961–1988.
- Anton Lozhkov, Raymond Li, Loubna Ben Allal, Federico Cassano, Joel Lamy-Poirier, Nouamane Tazi, Ao Tang, Dmytro Pykhtar, Jiawei Liu, Yuxiang Wei, Tianyang Liu, Max Tian, Denis Kocetkov, Arthur Zucker, Younes Belkada, Zijian Wang, Qian Liu, Dmitry Abul Khanov, Indraneil Paul, and 47 others. 2024. [StarCoder 2 and The Stack v2: The Next Generation](#). *Preprint*, arXiv:2402.19173.
- Mike A Merrill, Alexander G Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E Kelly Buchanan, and 1 others. 2026. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. *arXiv preprint arXiv:2601.11868*.
- OpenAI. 2026. GPT-5.5 System Card. <https://openai.com/index/gpt-5-5-system-card/>. Accessed: 2026-06-17.
- Dangfeng Pan, Zhensu Sun, Cenyuan Zhang, David Lo, and Xiaoning Du. 2026. [The hidden cost of readability: How code formatting silently consumes your LLM budget](#). In *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering*, ICSE '26. Distinguished Paper Award.
- Yuhao Qing, Boyu Zhu, Mingzhe Du, Zhijiang Guo, Terry Yue Zhuo, Qianru Zhang, Jie M Zhang, Heming Cui, Siu-Ming Yiu, Dong Huang, See-Kiong Ng, and Luu Anh Tuan. 2025. Effibench-x: A multi-language benchmark for measuring efficiency of llm-generated code. *Advances in neural information processing systems*.
- Qwen Team. 2026. [Qwen3.6-35B-A3B: Agentic coding power, now open to all](#).
- Mohamad Salim, Jasmine Latendresse, Sayed Hassan Khatoonabadi, and Emad Shihab. 2026. Tokenomics: Quantifying where tokens are used in agentic software engineering. *arXiv preprint arXiv:2601.14470*.
- Aman Sharma, Sushrut Thorat, and Paras Chopra. 2026. [Frontier coding agents use metaprogramming to adapt to unfamiliar programming languages](#). *Preprint*, arXiv:2606.10933.
- Lola Solovyeva and Fernando Castor. 2026. [Babbling suppression: Making LLMs greener one token at a time](#). *Preprint*, arXiv:2604.06755.
- Marilyn Strathern. 1997. ‘improving ratings’: audit in the british university system. *European Review*, 5(3):305–321.
- Zhensu Sun, Chengran Yang, Xiaoning Du, Zhou Yang, Li Li, and David Lo. 2025. [Token sugar: Making source code sweeter for LLMs through token-efficient shorthand](#). In *2025 IEEE/ACM 40th International Conference on Automated Software Engineering*, ASE '25, pages 2440–2451.
- Yuan-An Xiao, Pengfei Gao, Chao Peng, and Yingfei Xiong. 2026. Reducing cost of llm agents with trajectory reduction. *Proceedings of the ACM on Software Engineering*, 3.
- Bruce Yang, Xinfeng He, Huan Gao, Yifan Cao, Xiaofan Li, and David Hsu. 2025a. Codeagents: A token-efficient framework for codified multi-agent reasoning in llms. *arXiv preprint arXiv:2507.03254*.
- John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems*, 37:50528–50652.
- John Yang, Kilian Lieret, Carlos E. Jimenez, Alexander Wettig, Kabir Khandpur, Yanzhe Zhang, Binyuan Hui, Ofir Press, Ludwig Schmidt, and Diyi Yang. 2025b. [Swe-smith: Scaling data for software engineering agents](#). *Preprint*, arXiv:2504.21798.
- Daoguang Zan, Zhirong Huang, Wei Liu, Hanwu Chen, Shulin Xin, Linhao Zhang, Qi Liu, Li Aoyan, Lu Chen, Xiaojian Zhong, and 1 others. 2026. Multi-swe-bench: A multilingual benchmark for issue resolving. *Advances in Neural Information Processing Systems*, 38.
- Shaoqiu Zhang, Yuhang Wang, Jialiang Liang, Yuling Shi, Wenhao Zeng, Maoquan Wang, Shilin He, Ningyuan Xu, Siyu Ye, Kai Cai, and 1 others. 2026. Swe-explore: Benchmarking how coding agents explore repositories. *arXiv preprint arXiv:2606.07297*.
- Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2024. [SGLang: Efficient execution of structured language model programs](#). In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.
- Qixing Zhou, Jiacheng Zhang, Haiyang Wang, Rui Hao, Jiahe Wang, Minghao Han, Yuxue Yang, Shuzhe Wu, Feiyang Pan, Lue Fan, and 1 others. 2026. Featurebench: Benchmarking agentic coding for complex feature development. *arXiv preprint arXiv:2602.10975*.

A Hyperparameters and Configurations

We run open-weight models containers on a server with a Xeon Gold 6342 CPU, H100 GPUs, and 1TB RAM. We serve models locally with vLLM(Kwon et al., 2023), with a 128K context length. We use the official FP8 checkpoint for Qwen 3.6. We use the model’s recommended sampling parameters. Gemma 4: temperature 1.0, top- p 0.95, top- k 64; Qwen 3.6: temperature 1.0, top- p 0.95, top- k 20. We impose a 6K token reasoning budget on both models. For GLM, we use the HuggingFace Inference API with temperature 1.0, top- p 0.95, high-effort reasoning, and a maximum output length of 11,000 tokens per turn (covering both reasoning and visible output). Finally, we run GPT-5.5 with ‘medium’ reasoning, and run CLAUDE-SONNET-4.6 with reasoning enabled and a 10K token reasoning budget. Reproducing our full experiment (400 tasks per model) requires approximately 28 GPU-hours for Gemma and 19 GPU-hours for Qwen on H100s if run sequentially (request-level concurrency can reduce this substantially), and approximately \$100 for GPT-5.5, \$150 for Sonnet, and \$25 for GLM at standard API rates as of July 2026.

B Do Agents Cheat on MiniLCB?

A natural concern with visible tests is that agents may cheat by hard-coding expected outputs rather than solving the problem. To check for this, we task an LLM judge to flag suspicious solutions from the problem and final code, and we manually inspect every flagged case. Across all problems, languages, and models we find only two instances, both from GPT-5.5 on the same problem instance: the model derives a nearly-correct rule, hits one visible test that contradicts it, and adds an exact-match guard for that case. Cheating is therefore negligible and does not affect our analysis.

C Snapshot Span Label Distributions

Tables 5 and 6 give the full label breakdowns for Gemma discussed in §4.3 and §4.4. Table 5 shows *fix bug* dominating OCaml’s *stuck*→*breakthrough* escapes, versus a spread across *implement*, *execute plan*, and *optimize perf* on Python. Table 6 shows Gemma spends turns looping on STAY doing *cosmetic refactor* and *optimize perf*—unnecessary polishing under the benchmark’s design.

D Proprietary Model Results

Table 7 reports the mixed-effects model for the proprietary models. As with the open-weight models, every language incurs a significant output-token cost penalty relative to Python after controlling for problem difficulty.

The higher token costs for non-Python languages are reflected in the trajectory patterns shown in Figure 3, the GPT-5.5 counterpart to Figure 2 in the main text. On Python, GPT-5.5 almost always reaches a correct solution on the first write (*start*→*breakthrough*: 90), with almost no *stuck* activity. On OCaml, it enters *stuck* more often (*start*→*stuck*: 27) and accumulates 13 stuck self-loops before recovering.

Tables 8 and 9 show that proprietary models follow a much more direct workflow than the open-weight models. Most trajectories consist of a single implementation phase followed by verification, indicating that the models typically write the solution once, immediately run `./test.sh`, and submit if all tests pass. Despite this streamlined execution, proprietary models still externalize substantial reasoning into code. For example, on `arc194_a`, GPT-5.5 first writes a draft `solution.py` containing only commented-out derivations and a pass, then overwrites it with a complete implementation in the same turn. Similarly, Sonnet sometimes reasons in separate scratch files, solving one problem in a 700-line `think.py` before writing the final Rust solution. Proprietary models also frequently use Python as a reasoning language before translating to the target language. On `arc194_e` in OCaml, Sonnet spends 23 turns exploring the problem in Python, writing a series of brute-force scripts to discover and verify the invariant before producing an OCaml solution that passes all 32 tests on the first write. Likewise, GPT-5.5 often embeds Python directly via `python3 -c` or heredocs to prototype algorithms or validate invariants, especially for OCaml (27% of problems), Rust (19%), and Java (18%). These Python executions range from quick scratch calculations to full solution prototypes before translation. This behavior becomes more frequent on harder problems, suggesting that models adapt their workflow when additional algorithmic exploration is required.

E Measuring Agents’ Token Usage

For any nontrivial problem that takes more than one turn to solve, the agent’s conversation history

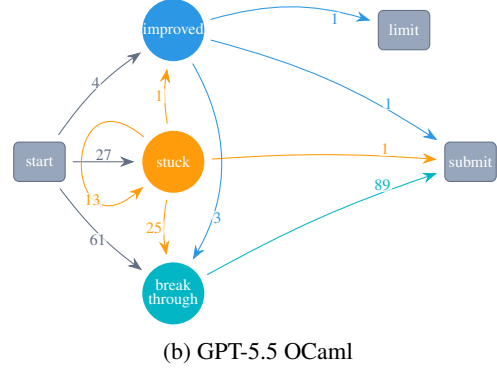
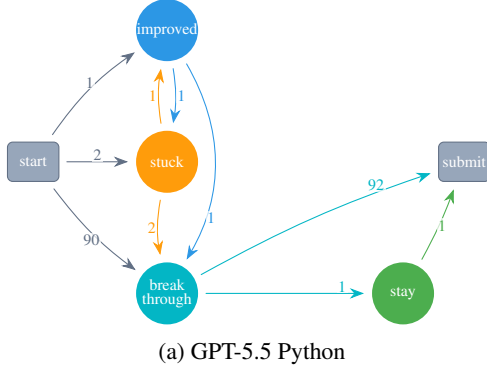


Figure 3: Solution-snapshot transition graphs for GPT-5.5, using the same node and edge semantics as Figure 2.

Label	Python ($n = 12$)	OCaml ($n = 53$)
<i>implement</i>	17%	4%
<i>rewrite</i>	—	4%
<i>execute plan</i>	25%	6%
<i>fix bug</i>	33%	60%
<i>debug failure</i>	—	6%
<i>infra confusion</i>	—	9%
<i>optimize perf</i>	25%	—
<i>cosmetic refactor</i>	—	11%

Table 5: Label distribution on edges from *stuck* to *break-through* for Gemma. OCaml breakthroughs are primarily due to *fix bug*.

Label	% of edges into STAY
<i>cosmetic refactor</i>	34%
<i>optimize perf</i>	22%
<i>implement</i>	18%
<i>verify</i>	8%
<i>fix bug</i>	5%
<i>revert</i>	4%
<i>edge case</i>	3%
<i>explore</i>	2%
<i>debug failure</i>	2%
<i>infra confusion</i>	1%
<i>execute plan</i>	1%
<i>rewrite</i>	1%

Table 6: Label distribution across all edges entering STAY for Gemma on Python ($n=258$). Over half are cosmetic refactoring or performance micro-optimization—changes that are unnecessary under the benchmark’s stated objective.

Table 7: Mixed-effects model results for GPT-5.5 and Sonnet: All three languages incur a significant output-token cost penalty relative to Python after controlling for problem difficulty.

Model	Contrast	Extra toks	Significance
GPT-5.5	Java vs. Python	$1.30\times$	$p < 0.001$
	Rust vs. Python	$1.36\times$	$p < 0.001$
	OCaml vs. Python	$1.30\times$	$p < 0.001$
	ICC	0.94	—
Sonnet	Java vs. Python	$1.18\times$	$p < 0.001$
	Rust vs. Python	$1.16\times$	$p < 0.001$
	OCaml vs. Python	$1.28\times$	$p < 0.001$
	ICC	0.97	—

Table 8: We run all intermediate solutions that the agent generates and report the number of tokens (turns) to the first solution that passes all tests. Proprietary models tend to submit their solution directly.

Model	Language	Tokens (Turns) to Solution	Tokens (Turns) after Solution
GPT-5.5	Python	601 (2)	42 (1)
	Java	841 (2)	42 (1)
	Rust	782 (3)	43 (1)
	OCaml	910 (2)	44 (1)
Sonnet	Python	878 (2)	103 (1)
	Java	1149 (1)	95 (1)
	Rust	1134 (1)	136 (2)
	OCaml	1484 (2)	136 (1)

is fed back into the model, turn after turn. So output tokens become input tokens, and the total number of input tokens over several turns can grow enormous. However, on a good inference platform, input processing is much cheaper than generating outputs, especially when the platform supports prefix caching (Zheng et al., 2024). When an agent works autonomously to solve a problem over several turns, prefix caching is very effective, and what matters most is the time required to generate outputs. This is why, like many others, we report

Model	Lang	Implement	Explore	Rewrite	Execute plan	Fix bug	Debug failure	Infra confusion	Revert	Verify	Edge case	Optimize perf.	Cosmetic refactor
GPT-5.5	java	49%	—	—	—	2%	1%	1%	—	49%	—	—	—
	ocaml	41%	1%	—	—	7%	—	11%	—	39%	—	—	—
	python	48%	—	1%	1%	2%	—	1%	—	48%	—	1%	—
	rust	49%	—	1%	—	1%	—	1%	—	49%	—	—	—
Sonnet	java	45%	1%	2%	—	7%	1%	—	—	44%	—	—	—
	ocaml	45%	1%	2%	—	8%	—	—	—	42%	—	—	—
	python	45%	4%	1%	—	2%	1%	1%	—	41%	—	3%	—
	rust	47%	1%	1%	—	4%	1%	1%	—	45%	—	—	—

Table 9: Percentage of spans assigned each label, for every (model, language) pair. GPT-5.5 and Sonnet are mostly *implement + verify*; debugging labels concentrate in OCaml; optimizations concentrate in python.

the total number of output tokens generated over a single conversation.

With contemporary reasoning models, the output tokens include both the reasoning and normal output. In this paper, we work with both closed and open-weight and reasoning models. The closed models that we use, from OpenAI and Anthropic, do not expose the reasoning output to users, but users are still charged for the reasoning tokens produced. From the perspective of the user, output tokens cost the same, whether or not the output is visible to the user.

F System Prompt

What follows is the template for the agent’s prompt. We substitute the template variables (italicized) with the problem text and the language configuration.

System: You are a competitive programming solver that can interact with a computer shell to solve programming tasks.

You are operating in an environment where you will think and issue AT LEAST ONE command, the system executes the command(s) in a subshell, you see the result, then you think and issue your next command(s).

****CRITICAL REQUIREMENTS:****

- Your response **SHOULD** include reasoning text explaining what you’re doing
- Your response **MUST** include AT LEAST ONE bash tool call. You can make MULTIPLE tool calls in a single response when the commands are independent (e.g., reading multiple files).

Task Instructions

Environment

- Your working directory contains a tests/ directory, with test cases (0-indexed)
- the input is tests/input_N.txt and the expected output is tests/output_N.txt
- The problem statement is provided below in this message.
- Write your solution to: *FILENAME*

- Your solution will be compiled and run as: *COMPILE_AND_RUN*
- A test runner is provided at ./test.sh. It runs the solution against every visible test, diffs the output, and prints PASS or FAIL with each test input/output filename.
- To test your program, run exactly: ./test.sh
- These are the only tests I care about. If ./test.sh exits with status 0, the solution passes the tests; empty diff output means success.

Submission

When ./test.sh exits with status 0, immediately submit by running: echo COMPLETE_TASK_AND_SUBMIT_FINAL_OUTPUT

User: Solve the following competitive programming problem in *PROGRAMMING_LANGUAGE*.

Programming Problem:

PROBLEM